C Data Types

Understanding the fundamental elements of C programming: data types, variables, and constants

Introduction

Computer programs usually work with different types of data and need a way to store the values being used. These values can be numbers. C language has two ways of storing number values—Data types and Variables—with many options for each.

Fundamental Elements

Data types and variables are the fundamental elements of each program. Simply speaking, a program is nothing else than defining them and manipulating them.

Variable

A data storage location that has a value that can change during program execution.

Constant

Has a fixed value that can't change.

Unit Focus

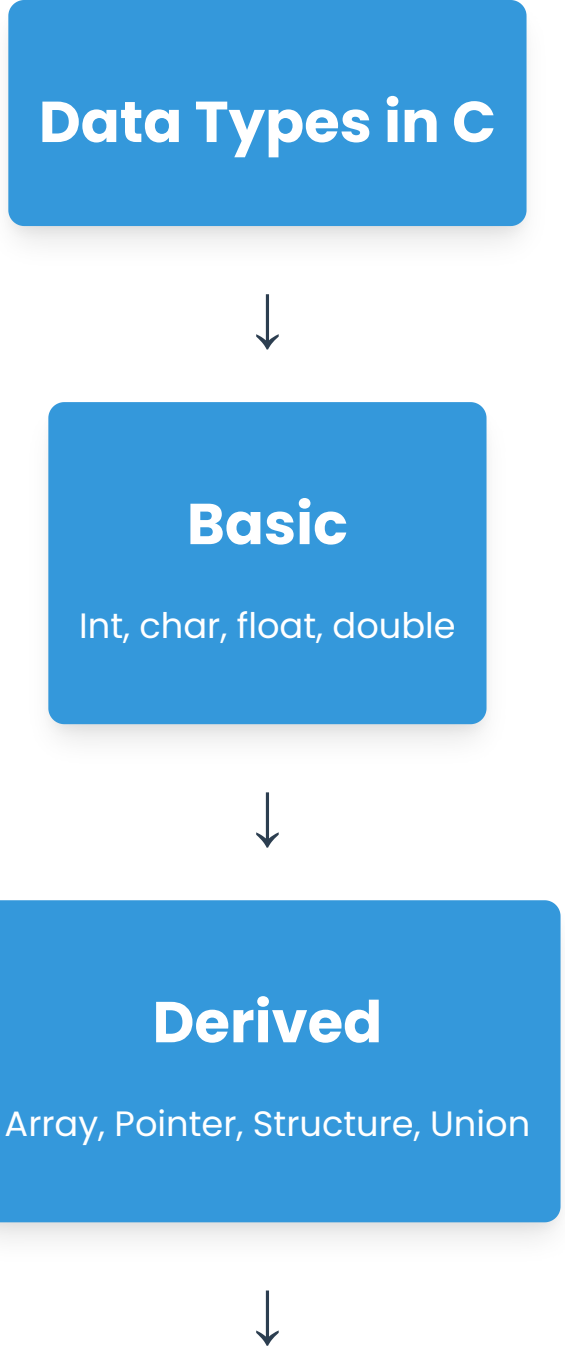
This unit is concerned with the basic elements used to construct simple C program statements. These elements include the C character set, identifiers and keywords, data types, constants, variables and arrays, declaration and naming conventions of variables.

💡 **Key Takeaway**

This unit will explain to you the fundamental elements of language C.

Data Types in 'C'

Each variable in C has an associated data type. It specifies the type of data that the variable can store like integer, character, floating, double, etc. Each data type requires different amounts of memory and has some specific operations which can be performed over it.


Data Types in C
Basic
Int, char, float, double
Derived
Array, Pointer, Structure, Union

User Defined

Class, Structure, Union

↓

Enumeration

Enum

↓

Void

Function, Reference

Primitive Data Types

Most basic data types for simple values

User Defined Data Types

Defined by the user himself

Derived Types

Derived from primitive types

Primitive Data Types (Basic Data Types)

The basic data types are integer-based and floating-point based. C language supports both signed and unsigned literals.

The memory size of the basic data types may change according to 32 or 64-bit operating system.

Data Types	Memory Size	Range
Char	1 byte	-128 to 127
Signed char	1 byte	-128 to 127

Data Types	Memory Size	Range
Unsigned char	1 byte	0 to 255
Short	2 byte	-32,768 to 32,767
Signed short	2 byte	-32,768 to 32,767
Unsigned short	2 byte	0 to 65,535
Int	2 byte	-32,768 to 32,767
Signed int	2 byte	-32,768 to 32,767
Unsigned int	2 byte	0 to 65,535
Short int	2 byte	-32,768 to 32,767
Signed short int	2 byte	-32,768 to 32,767
Unsigned short int	2 byte	0 to 65,535
Long int	4 byte	-2,147,483,648 to 2,147,483,647
Signed long int	4 byte	-2,147,483,648 to 2,147,483,647

Data Types	Memory Size	Range
Unsigned long int	4 byte	0 to 4,294,967,295
Float	4 byte	1.2E-38 to 3.4E+38
Double	8 byte	2.3E-308 to 1.7E+308
Long double	10 byte	3.4E-4932 to 1.1E+4932

💡 **Note**

Different data types also have different ranges up to which they can store numbers. These ranges may vary from compiler to compiler.

Integer Types

Integers are entire numbers without any fractional or decimal parts, and the int data type is used to represent them.

It is frequently applied to variables that include values, such as counts, indices, or other numerical numbers. The int data type may represent both positive and negative numbers because it is signed by default.

An int takes up 4 bytes of memory on most devices, allowing it to store values between around -2 billion and +2 billion.

Data Type Qualifiers

Short, long, signed, unsigned are called the data type qualifiers and can be used with any data type.

Memory Usage

A short int requires less space than int and long int may require more space than int.

Unsigned Types

Unsigned bits use all bits for magnitude; therefore, this type of number can be larger.

```c
// Example to get the size of int type
#include <stdio.h>
#include <limits.h>

int main() {
```

```
  printf("Storage size for int: %d \n", sizeof(int));
  return 0;
}
```

```
Storage size for int: 4
```

> 💡 **Tip**
>
> To get the exact size of a type or a variable on a particular platform, you can use the sizeof operator.

Character and Floating-Point Types

Character Type

```
char grade = 'A';
```

Floating-Point Types

Double Type

```
double pi = 3.14159265359;
```

✅

1 byte, -128 to 127

```
float temperature = 98.6;
```

✅

4 bytes, 6 decimal places

✅

8 bytes, 15 decimal places

Type	Storage Size	Value Range	Precision
Char	1 byte	ASCII character set (-128 to 127)	-
Unsigned char	1 byte	ASCII character set (0 to 255)	-
Float	4 byte	1.2E-38 to 3.4E+38	6 decimal places
Double	8 byte	2.3E-308 to 1.7E+308	15 decimal places
Long double	10 byte	3.4E-4932 to 1.1E+4932	19 decimal places

💡 **Note**

The header file float.h defines macros that allow you to use these values and other details about the binary representation of real numbers in your programs.

Derived Data Type

Beyond the fundamental data types, C also supports derived data types, including arrays, pointers, structures, and unions. These data types give programmers the ability to handle heterogeneous data, directly modify memory, and build complicated data structures.

Array

An array, a derived data type, lets you store a sequence of fixed-size elements of the same type. It provides a mechanism for joining multiple targets of the same data under the same name.

numbers[5] = {10, 20, 30, 40, 50}

```c
// Example of declaring and utilizing an array
#include <stdio.h>

int main() {
  int numbers[5]; // Declares an integer array with a size of 5 elements
  // Assign values to the array elements
```

```
  numbers[0] = 10;
  numbers[1] = 20;
  numbers[2] = 30;
  numbers[3] = 40;
  numbers[4] = 50;
  // Display the values stored in the array
  printf("Values in the array: ");
  for (int i = 0; i < 5; i++) {
    printf("%d ", numbers[i]);
  }
  printf("\n");
  return 0;
}
```

```
Values in the array: 10 20 30 40 50
```

💡 **Array Characteristics**

The index is used to access the elements of the array, with a 0 index for the first entry. The size of the array is fixed at declaration time and cannot be changed during program execution.

The void Type

The void type specifies that no value is available. It is used in three kinds of situations:

Function returns as void

Functions that do not return any value

Function arguments as void

Functions that do not accept any parameter

Pointers to void

Represents the address of an object, but not its type

S. No.	Type and Description	
1	**Function returns as void**	Functions that do not return any value or you can say they return void. A function with no return value has the return type as void.
2	**Function arguments as void**	Functions that do not accept any parameter. A function with no parameter can accept a void.
3	**Pointers to void**	A pointer of type void * represents the address of an object, but not its type.

```
// Example of void function
void exit(int status);
```

```c
// Example of void parameter
int rand(void);
```

```c
// Example of void pointer
void *malloc(size_t size);
```


Lvalues and Rvalues in C

There are two kinds of expressions in C:

lvalue

Expressions that refer to a memory location are called "lvalue" expressions. An lvalue may appear as either the left-hand or right-hand side of an assignment.

→

rvalue

The term rvalue refers to a data value that is stored at some address in memory. An rvalue is an expression that cannot have a value assigned to it which means an rvalue may appear on the right-hand side but not on the left-hand side of an assignment.

Valid vs Invalid Statements

Variables are lvalues and so they may appear on the left-hand side of an assignment. Numeric literals are rvalues and so they may not be assigned and cannot appear on the left-hand side.

Valid

```
int g = 20;
```

Invalid

```
10 = 20;
```

⚠ Compilation Error

Attempting to assign a value to an rvalue will generate a compile-time error.

Summary

Data Types

C provides various data types to store different kinds of data

Primitive Types

Basic types like int, char, float, double

Derived Types

Arrays, pointers, structures, unions

Lvalues & Rvalues

Understanding the difference between assignable and non-assignable expressions

Thank you for your attention!